

RELATÓRIO TÉCNICO

Projeto Assistente de Estudos Inteligente

Assistente de estudos baseado em RAG
Funcionamento, arquitetura e requisitos para execução

Documentação gerada a partir do material do projeto.

1. Visão geral

O projeto é um **Assistente de Estudos Inteligente**. O aluno faz perguntas em linguagem natural sobre um material de aula (um PDF), e o sistema responde **em português**, baseando-se **exclusivamente no conteúdo daquele PDF** — não inventa nada de fora.

A técnica central é o **RAG (Retrieval-Augmented Generation)**: em vez de jogar o PDF inteiro no modelo de IA, o sistema primeiro **busca os trechos mais relevantes** do documento e só então pede ao modelo que **gere a resposta** com base nesses trechos. Isso deixa as respostas mais precisas e baratas.

A arquitetura é de **microsserviços**: quatro programas independentes que conversam entre si por HTTP, mais o modelo de IA rodando na nuvem (Google Colab).

2. Como funciona (fluxo de uma pergunta)

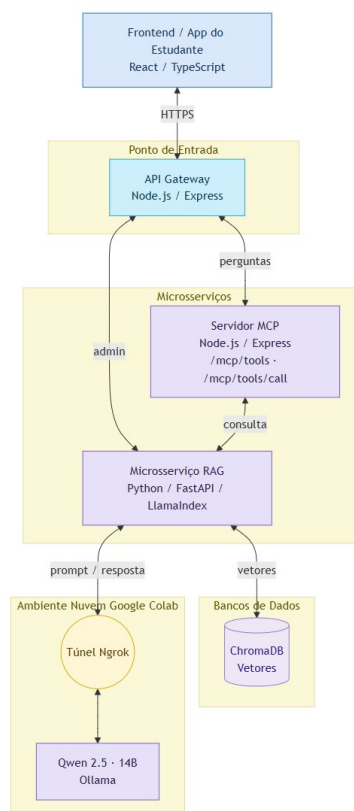


Figura 1 — Arquitetura de microsserviços do sistema.

Passo a passo do que acontece quando alguém pergunta algo:

- **Frontend:** o aluno digita a pergunta; ela é enviada via POST para o Gateway em `127.0.0.1:8000/perguntar`.
- **API Gateway:** é o único ponto de entrada. Ele repassa a pergunta ao Servidor MCP no formato do protocolo MCP (tool: `consultar_documentos_aula`). Também expõe rotas administrativas (`/health`, `/debug`, `/reindexar`) que vão direto ao RAG.

- **Servidor MCP:** implementa o MCP (Model Context Protocol) sobre HTTP. Ele “descobre” e “executa” ferramentas. A ferramenta `consultar_documentos_aula` delega a busca ao RAG.
- **Microsserviço RAG** — onde está a inteligência (detalhado abaixo).
- **Modelo de IA (LLM):** o RAG não roda o modelo localmente. Ele se conecta ao Ollama rodando dentro do Google Colab (que tem GPU potente), exposto à internet por um túnel ngrok. A URL fica no arquivo `.env` e muda toda vez que o Colab reinicia.

O que o Microsserviço RAG faz por dentro

- **Indexação (na primeira execução):** lê o PDF da pasta `documentos/` , quebra o texto em pedaços (“chunks”) de ~1024 caracteres com sobreposição de 128, gera os embeddings (vetores numéricos) localmente com o FastEmbed e salva tudo no ChromaDB. O índice é persistido em disco e reaproveitado depois. Um hash MD5 do PDF detecta se o material mudou e dispara reindexação automática.
- **Busca híbrida:** combina busca vetorial (semântica, via ChromaDB) com BM25 (busca por palavras-chave), fundindo os resultados com RRF (Reciprocal Rank Fusion). Pega tanto sinônimos quanto termos exatos.
- **Geração:** monta um prompt rígido (sempre responder em PT-BR, não inventar, citar o material) e envia, junto com os melhores trechos, ao modelo Qwen 2.5.
- **Ficha técnica dinâmica:** extrai automaticamente título e autor do PDF para o Assistente de Estudos Inteligente saber responder “quem é o autor / qual o nome do material”.
- **Cache:** guarda as últimas 500 respostas por 1 hora, respondendo perguntas repetidas instantaneamente.

3. Tecnologias usadas

Camada	Tecnologia	Função
Frontend	React 19 + TypeScript + Vite	Interface de chat no navegador
API Gateway	Node.js + Express	Porta de entrada / roteador
Servidor MCP	Node.js + Express	Expõe a busca como ferramenta MCP
Microsserviço RAG	Python + FastAPI + Uvicorn	Orquestra busca e geração
Framework RAG	LlamaIndex	Indexação, chunking, retrieval, prompts
Banco de vetores	ChromaDB	Armazena os embeddings dos trechos
Embeddings	FastEmbed (multilingual-e5-large)	Transforma texto em vetores (local)
Busca por palavra	BM25 + RRF	Complementa a busca vetorial
Modelo de IA (LLM)	Qwen 2.5 14B via Ollama	Gera as respostas
Infra do modelo	Google Colab (GPU)	Roda o Ollama na nuvem
Túnel de rede	ngrok	Expõe o Colab para a internet
Comunicação	HTTP / REST + protocolo MCP	Conversa entre os serviços

4. O que é preciso para rodar

Contas e programas

- **Python 3.10+** (no Windows, marcar “Add Python to PATH”).
- **Node.js 18 LTS+** (já vem com o npm).
- **Conta no Google** (para o Google Colab).
- **Conta no ngrok** com authtoken (para o túnel).
- **GPU recomendada** para os embeddings locais (no ambiente original, uma RTX 3050 de 4 GB — funciona, mas é o gargalo).

Instalação

```
# RAG (Python) – na pasta rag_service
pip install fastapi uvicorn httpx python-dotenv pydantic chromadb \
    cachetools pypdf openai llama-index llama-index-retrievers-bm25 \
    llama-index-llms-openai-like llama-index-embeddings-fastembed \
    llama-index-vector-stores-chroma

# Serviços Node (rodar npm install em cada pasta)
cd ../api_gateway ; npm install
cd ../mcp_service ; npm install
cd ../frontend ; npm install
```

Ligar o modelo no Google Colab (rodar as células na ordem)

- Instalar Ollama + pyngrok.
- Conferir a GPU (`!nvidia-smi`).
- Subir o Ollama e baixar o modelo (`ollama serve` + `ollama pull qwen2.5:14b`).
- Abrir o túnel ngrok (com seu authtoken) — ele devolve uma URL `https://...ngrok-free.dev` .

Configurar a URL em rag_service/.env

```
NGROK_URL=https://SEU-ENDERECO.ngrok-free.dev
```

Subir os serviços (4 terminais, nesta ordem)

```
cd rag_service ; python main.py      # Terminal 1 – RAG      (:8001)
cd mcp_service ; npm start            # Terminal 2 – MCP      (:8002)
cd api_gateway ; npm start            # Terminal 3 – Gateway  (:8000)
cd frontend ; npm run dev             # Terminal 4 – Frontend (:5173)
```

Depois é só abrir `http://localhost:5173` no navegador e usar.